

Lab Preliminary Report

Course: Cs 223 Digital
Design

Section: 2

Lab Number: 2

Name: Eralp Yiğit Boz

ID: 22403188

Date: 29.03.2026

Question 1

Multifunction Register

1.1) Synchronously Resettable D flip-flop.

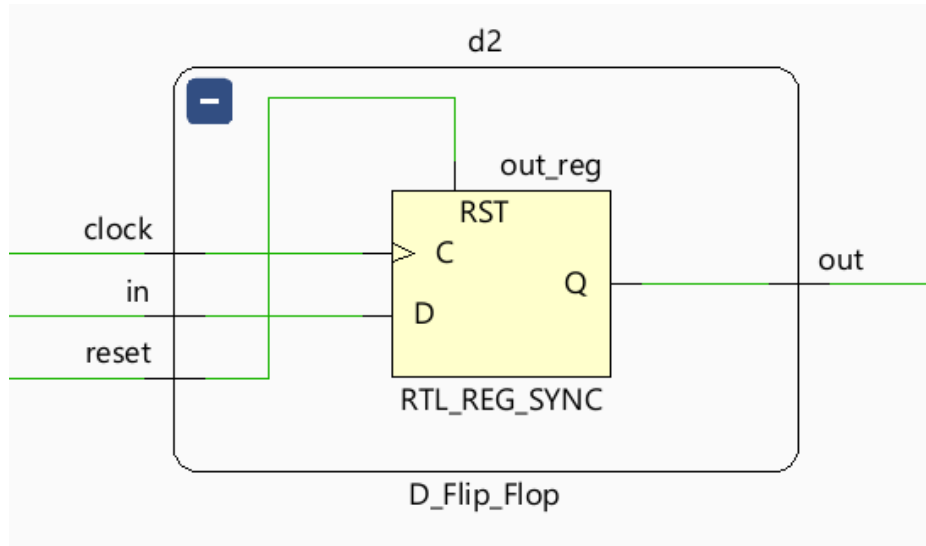


Figure 1 d flip flop schematic

SystemVerilog Module for D Flip Flop

```
module D_Flip_Flop(
    input logic clock,
    input logic reset,
    input logic in,
    output logic out
);

    always_ff @(posedge clock) begin
        if (reset) begin
            out <= 1'b0;
        end else begin
            out <= in;
        end
    end
end
```

end

endmodule

1.2) Circuit Schematic of Multifunction Register

In this design, an 8-bit structured multifunction register is created by connecting eight 8-to-1 multiplexers and synchronously resettable D flip-flops. The multiplexers act as combinational logic to select the desired register operation (such as shifting, rotating, or parallel loading) based on a 3-bit control signal, while the D flip-flops synchronously maintain and update the current 8-bit state in each clock cycle.

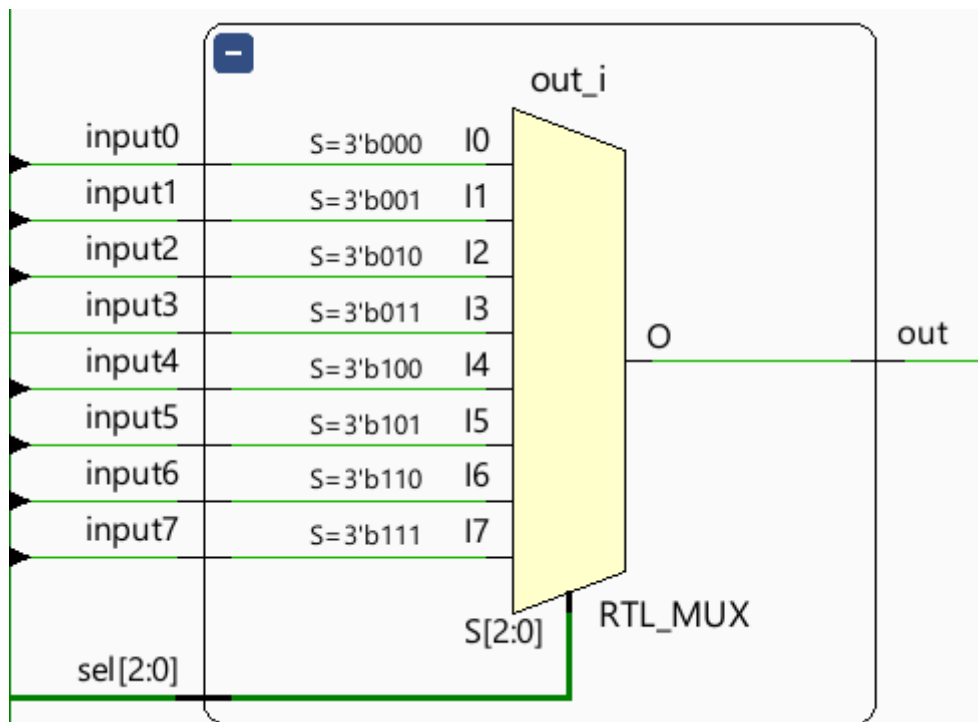


Figure 2 8 to 1 Mux

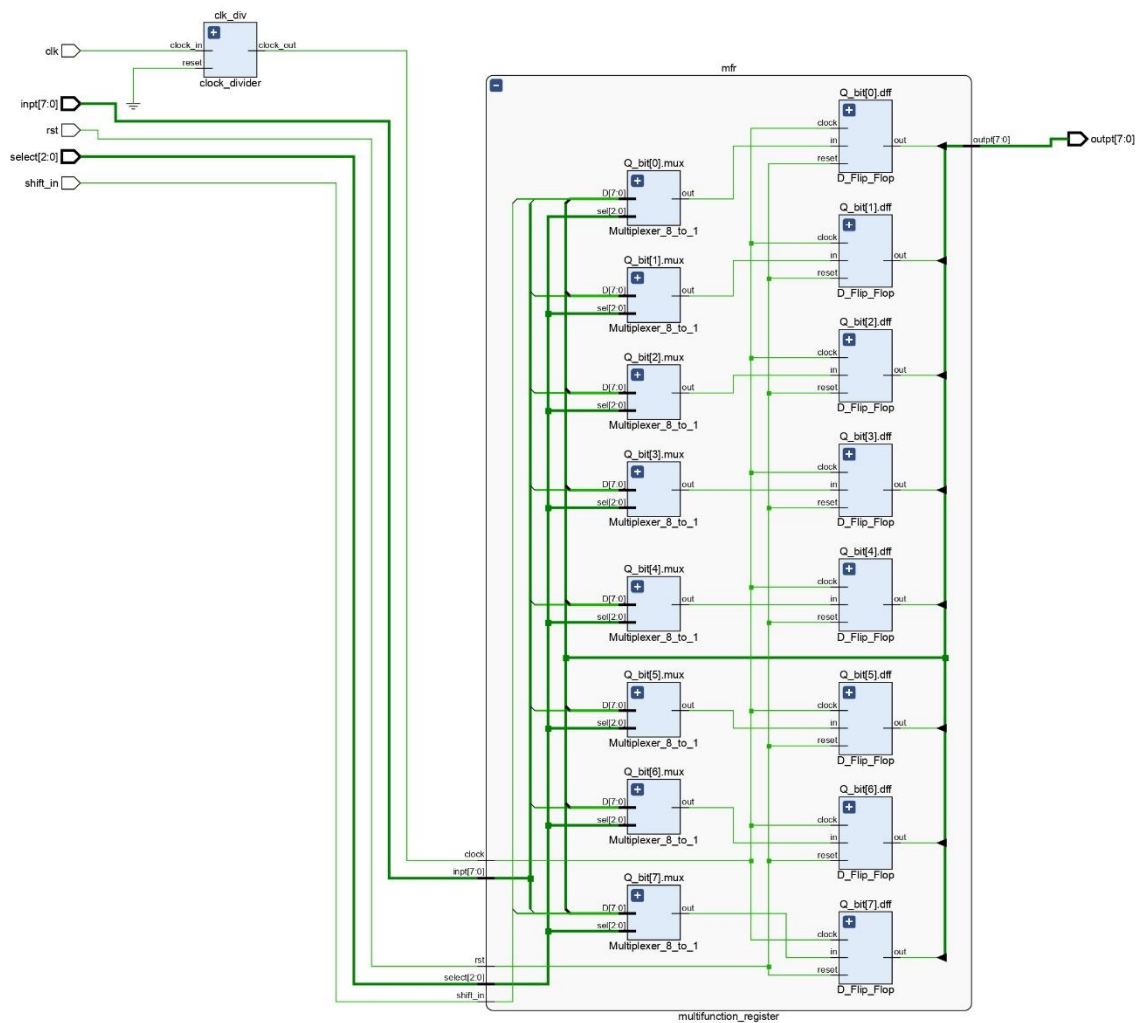


Figure 3 Multifunction Register

1.3) Systemverilog module and Testbench for Multifunction Register

Multiplexer SytemVerilog

```

module Multiplexer_8_to_1(
    input logic input0, input1, input2, input3, input4, input5, input6, input7,
    input logic [2:0] sel,
    output logic out
);

    always_comb begin
        case(sel)
            3'b000: out = input0;
            3'b001: out = input1;
            3'b010: out = input2;
            3'b011: out = input3;
            3'b100: out = input4;
            3'b101: out = input5;
            3'b110: out = input6;
            3'b111: out = input7;
            default: out = 1'b0;
        endcase
    end

endmodule

```

Multifunction Register SystemVerilog

```

module multifunction_register(
    input logic clock,
    input logic rst,

```

```

input logic shift_in,
input logic [2:0] select,
input logic [7:0] inpt,
output logic [7:0] outpt
);

```

```

logic mux_out_0, mux_out_1, mux_out_2, mux_out_3, mux_out_4,
mux_out_5, mux_out_6, mux_out_7;

```

```
//BIT 0
```

```

Multiplexer_8_to_1 m0 (.input0(outpt[0]), .input1(shift_in), .input2(outpt[1]),
.input3(inpt[0]), .input4(outpt[7]), .input5(outpt[1]), .input6(outpt[7]),
.input7(outpt[4]), .sel(select), .out(mux_out_0));

```

```
D_Flip_Flop d0 (.clock(clock), .reset(rst), .in(mux_out_0), .out(outpt[0]));
```

```
//BIT 1
```

```

Multiplexer_8_to_1 m1 (.input0(outpt[1]), .input1(outpt[0]),
.input2(outpt[2]), .input3(inpt[1]), .input4(outpt[0]), .input5(outpt[2]),
.input6(outpt[6]), .input7(outpt[5]), .sel(select), .out(mux_out_1));

```

```
D_Flip_Flop d1 (.clock(clock), .reset(rst), .in(mux_out_1), .out(outpt[1]));
```

```
//BIT 2
```

```

Multiplexer_8_to_1 m2 (.input0(outpt[2]), .input1(outpt[1]),
.input2(outpt[3]), .input3(inpt[2]), .input4(outpt[1]), .input5(outpt[3]),
.input6(outpt[5]), .input7(outpt[6]), .sel(select), .out(mux_out_2));

```

```
D_Flip_Flop d2 (.clock(clock), .reset(rst), .in(mux_out_2), .out(outpt[2]));
```

```
//BIT 3
```

```

    Multiplexer_8_to_1 m3 (.input0(outpt[3]), .input1(outpt[2]),
    .input2(outpt[4]), .input3(inpt[3]), .input4(outpt[2]), .input5(outpt[4]),
    .input6(outpt[4]), .input7(outpt[7]), .sel(select), .out(mux_out_3));

```

```

    D_Flip_Flop d3 (.clock(clock), .reset(rst), .in(mux_out_3), .out(outpt[3]));

```

```

//BIT 4

```

```

    Multiplexer_8_to_1 m4 (.input0(outpt[4]), .input1(outpt[3]),
    .input2(outpt[5]), .input3(inpt[4]), .input4(outpt[3]), .input5(outpt[5]),
    .input6(outpt[3]), .input7(outpt[0]), .sel(select), .out(mux_out_4));

```

```

    D_Flip_Flop d4 (.clock(clock), .reset(rst), .in(mux_out_4), .out(outpt[4]));

```

```

//BIT 5

```

```

    Multiplexer_8_to_1 m5 (.input0(outpt[5]), .input1(outpt[4]),
    .input2(outpt[6]), .input3(inpt[5]), .input4(outpt[4]), .input5(outpt[6]),
    .input6(outpt[2]), .input7(outpt[1]), .sel(select), .out(mux_out_5));

```

```

    D_Flip_Flop d5 (.clock(clock), .reset(rst), .in(mux_out_5), .out(outpt[5]));

```

```

//BIT 6

```

```

    Multiplexer_8_to_1 m6 (.input0(outpt[6]), .input1(outpt[5]),
    .input2(outpt[7]), .input3(inpt[6]), .input4(outpt[5]), .input5(outpt[7]),
    .input6(outpt[1]), .input7(outpt[2]), .sel(select), .out(mux_out_6));

```

```

    D_Flip_Flop d6 (.clock(clock), .reset(rst), .in(mux_out_6), .out(outpt[6]));

```

```

// BIT 7

```

```

    Multiplexer_8_to_1 m7 (.input0(outpt[7]), .input1(outpt[6]), .input2(shift_in),
    .input3(inpt[7]), .input4(outpt[6]), .input5(outpt[0]), .input6(outpt[0]),
    .input7(outpt[3]), .sel(select), .out(mux_out_7));

```

```

    D_Flip_Flop d7 (.clock(clock), .reset(rst), .in(mux_out_7), .out(outpt[7]));

```

```

endmodule

```

Testbench

```
module testbenchForRegister();
```

```
    logic clock;
```

```
    logic rst;
```

```
    logic shift_in;
```

```
    logic [2:0] select;
```

```
    logic [7:0] inpt;
```

```
    logic [7:0] outpt;
```

```
    multifunction_register dut (
```

```
        .clock(clock),
```

```
        .rst(rst),
```

```
        .shift_in(shift_in),
```

```
        .select(select),
```

```
        .inpt(inpt),
```

```
        .outpt(outpt)
```

```
    );
```

```
    always #5 clock = ~clock;
```

```
    initial begin
```

```
        clock = 0;
```

```
        rst = 1;
```

```
        shift_in = 0;
```

```
        select = 3'b000;
```

```
        inpt = 8'h00;
```



```
#10;

rst = 0;

//(1010 0101)


// 011 (Parallel Load)
select = 3'b011;
inpt = 8'hA5;
#10;


// 111 (Nibble Swap)
// A5 (1010 0101) -> 5A (0101 1010)
select = 3'b111;
#10;


// 110 (Bit Reversal)
select = 3'b110;
#10;


// 101 (Rotate Right)
select = 3'b101;
#10;


// 010 (Shift Right)
select = 3'b010;
shift_in = 1'b0;
#10;
```

```

// 100 (Rotate Left)
select = 3'b100;
#10;

// 001 (Shift Left)
select = 3'b001;
shift_in = 1'b1;
#10;

// 000 (Maintain Present Value)
select = 3'b000;
#10;

$finish;
end

```

```
endmodule
```

Question 2

Hexadecimal to 7-Segment Decoder

2.1) SystemVerilog and Testbench Code for 7-Segment Decoder

SystemVerilog

```

module hex_to_7seg(
    input logic [3:0] hex_val,
    output logic [6:0] segments
);

always_comb begin

```

```
case(hex_val)

    4'h0: segments = 7'b0000001; // 0
    4'h1: segments = 7'b1001111; // 1
    4'h2: segments = 7'b0010010; // 2
    4'h3: segments = 7'b0000110; // 3
    4'h4: segments = 7'b1001100; // 4
    4'h5: segments = 7'b0100100; // 5
    4'h6: segments = 7'b0100000; // 6
    4'h7: segments = 7'b0001111; // 7
    4'h8: segments = 7'b0000000; // 8
    4'h9: segments = 7'b0000100; // 9
    4'hA: segments = 7'b0001000; // A
    4'hB: segments = 7'b1100000; // b
    4'hC: segments = 7'b0110001; // C
    4'hD: segments = 7'b1000010; // d
    4'hE: segments = 7'b0110000; // E
    4'hF: segments = 7'b0111000; // F
    default: segments = 7'b1111111;
endcase

end

endmodule
```

Testbench

```
module tb_hex_to_7seg();
```

```
logic [3:0] hex_val;
```

```
logic [6:0] segments;
```

```
hex_to_7seg dut (  
    .hex_val(hex_val),  
    .segments(segments)  
);
```

```
initial begin
```

```
    for (int i = 0; i < 16; i++) begin
```

```
        hex_val = i;
```

```
        #10;
```

```
    end
```

```
    $finish; // Simülasyonu bitir
```

```
end
```

```
endmodule
```

2.2) Questions

1. What are the advantages/disadvantages of using hex values to represent numbers? List two or three points and explain them with a few short sentences. Provide at least one disadvantage.

Advantages:

- Hexadecimal values significantly reduce the length of numbers compared to the binary system, making it much easier for engineers to read, write, and debug data without making representation errors.
- Digital systems typically group data into 8-bit bytes or larger words. The hexadecimal system is perfectly suited to this hardware architecture because an 8-bit byte can always be accurately represented by exactly two hexadecimal digits.

Disadvantage:

- To display hexadecimal values on physical hardware (such as 7-segment displays), additional combinational logic circuits must be designed and implemented. These additional logic circuits increase the cost of the system.

2. Research how the full four-digit seven-segment display on the Basys3 board works. Think about which sequential components you need to drive it. Explain with a few short sentences.

The four digits on the Basys3 board physically share the same segment data lines. A technique called Time Division Multiplexing is used to display different characters simultaneously on each digit. To operate this, a sequential component is required, specifically a Counter (driven by a fast clock signal of approximately 1 kHz). This counter loops rapidly through the digits, turning on the anode of one digit at a time and sending the corresponding data. Finally, the 'off' detail in Table 2 means the display uses an Active Low configuration; that is, a logical '0' is required to turn on a segment, which explains the reverse logic shown in the table.